

Sveučilište J. J. Strossmayera u Osijeku
Fakultet primijenjene matematike i informatike

Autić na daljinsko upravljanje
Borna Ocvirk

SADRŽAJ

UVOD	3
OPIS SUSTAVA	4
LOGIKA PONAŠANJA SUSTAVA	5
TEHNIČKE SPECIFIKACIJE	6
Hardver	6
ESP32	6
28BYJ-48	7
ULN2003	8
HC-SR04	9
Softver	10
Xtensa GCC	10
ESP-IDF	10
IMPLEMENTACIJA	11
Programski dio sustava	11
Uporaba koračnih motora	11
Uporaba HC-SR04 senzora	12
Uporaba web sučelja	13
Implementacija logike sustava	13
Elektronička shema sustava	14
LITERATURA	15

UVOD

Prvi se puta pojavio kao prijevozno sredstvo za uzvišene ljude svijeta, a ubrzo nakon toga postao dostupan za sve. Od transporta velike količine robe i ljudi do činjenja svijeta manjim i dostupnijim nego ikad, automobil je kao izum jedna od onih stvari koja je unaprijedila i ubrzala razvoj čovječanstva u neviđenom pogledu. Kako je rasla potreba za automobilima, tako se ukazala prilika za predstavljanje navedenog izuma i mlađim uzrastima u obliku autića na daljinsko upravljanje. Bio je i ostao jedna od najprodavanijih igračaka za djecu svih uzrasta te mu je cilj zabava za navedene što upravo ovaj projekt i donosi.

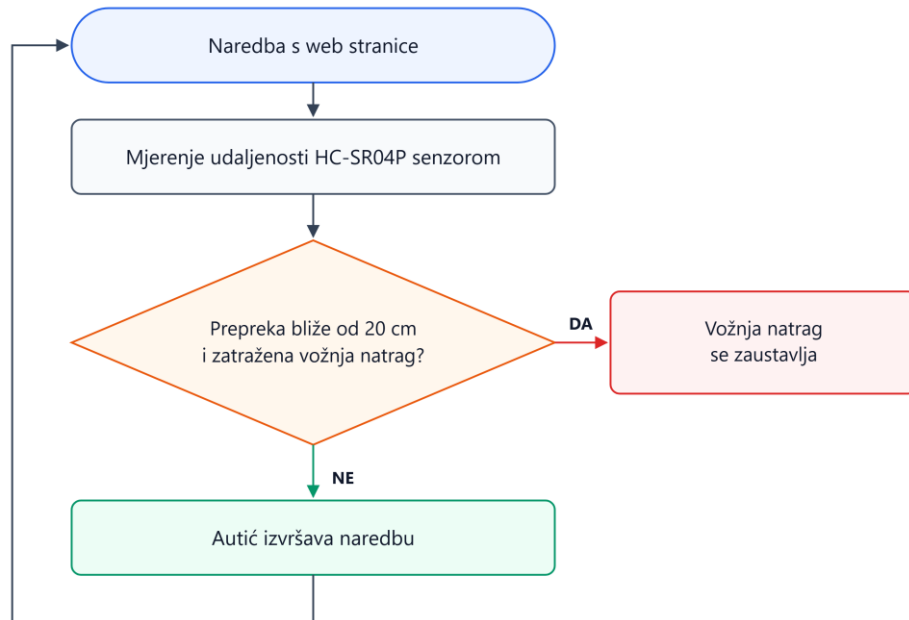
OPIS SUSTAVA

Glavni je dio sustava ESP32 devkit mikroupravljač koji u realnom vremenu dobavlja te obrađuje podatke prikupljene od senzora spojenog na njegove pinove. Specifični model navedenog senzora je HC-SR04 ultrazvučni senzor za mjerenje udaljenosti od stražnjih prepreka.

Kretanje autića izvode dva 28BYJ-48 koračna motora, od kojih svaki ima vlastiti ULN2003 upravljački modul. Motori su postavljeni na suprotne strane šasijske te se skretanje ne izvodi zakretanjem prednjih kotača, nego razlikom smjerova lijevog i desnog kotača. Upravljanje je prebačeno na mobitel: ESP32 podiže vlastitu Wi-Fi pristupnu točku i na njoj posluhuje web stranicu s tipkama za vožnju, na kojoj se prikazuju i stanje pogona, izmjerena udaljenost i prijeđeni put. Cijeli se sustav napaja iz jednog USB power banka priključenog na ESP32 pločicu, iz koje se napon dalje razvodi prema upravljačkim modulima i senzoru.

LOGIKA PONAŠANJA SUSTAVA

Sustav je isprogramiran tako da naredbu korisnika provjeri prije nego što je izvrši. Logika koju pri tome slijedi prikazana je dijagramom ispod.



Slika 1 - Logika sustava

TEHNIČKE SPECIFIKACIJE

Hardver

ESP32

ESP32 je razvojna pločica bazirana na ESP32-WROOM-32 mikroupravljaču. Sadrži dvojezgreni 32-bitni procesor, ugrađenu podršku za Wi-Fi i Bluetooth te niz ulazno-izlaznih pinova s podrškom za PWM, ADC, DAC i komunikacijske protokole I2C, SPI i UART. Za povezivanje s računalom koristi CH340C pretvarač.



Slika 2 - ESP32

28BYJ-48

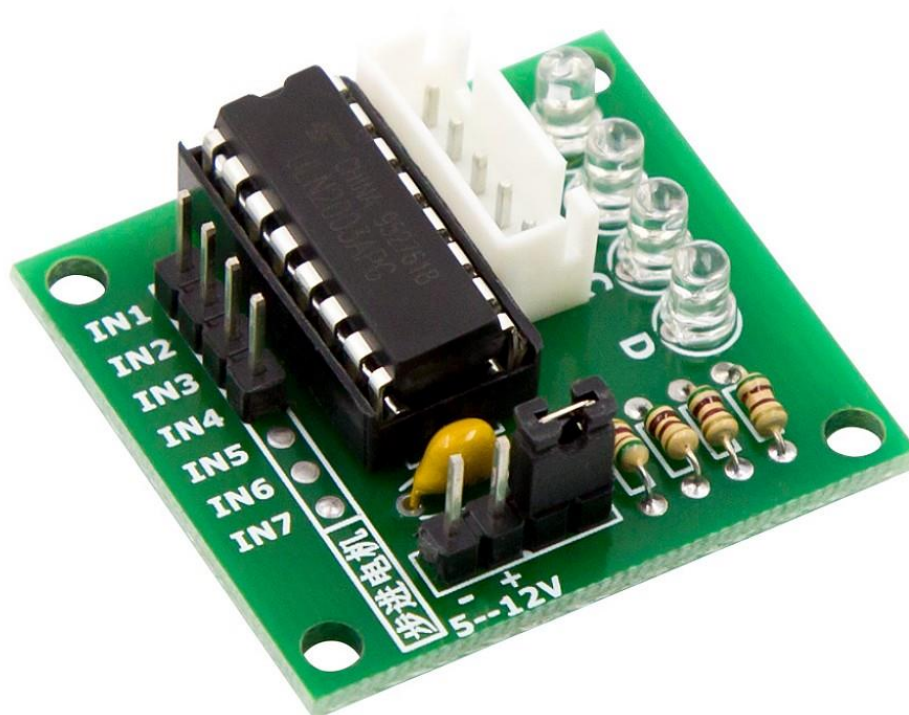
28BYJ-48 je mali unipolarni koračni motor s ugrađenim reduktorom, namijenjen napajanju od pet volti. Za puni okret izlazne osovine u polukoračnom načinu rada potrebno je 4096 impulsa, pa je motor spor, ali precizan i lako se upravlja bez povratne veze jer položaj osovine slijedi iz broja poslanih impulsa.



Slika 3 - 28BYJ-48

ULN2003

ULN2003 je niz Darlingtonovih parova koji služi kao pojačalo struje između mikroupravljača i motora. ESP32 na ulaze IN1 do IN4 daje samo logički signal od 3,3 volta, dok struju namota modul vuče iz vlastitog napajanja od pet volti. Zbog toga se koračni motor nikada ne spaja izravno na pinove mikroupravljača.



Slika 4 - ULN2003

HC-SR04

HC-SR04 je ultrazvučni senzor koji udaljenost mjeri vremenom leta zvučnog vala. Na pin TRIG dovodi se impuls od deset mikrosekundi, nakon čega senzor drži pin ECHO u visokom stanju onoliko dugo koliko je zvuku trebalo do prepreke i natrag, pa se udaljenost u centimetrima dobiva dijeljenjem tog trajanja s brojem 58. Doseg mu je od dva do četristo centimetara. Za razliku od običnog HC-SR04, P inačica radi na 3,3 volta, pa se ECHO spaja izravno i djeliteľ napona nije potreban.



Slika 5 - HC-SR04P

Softver

Xtensa GCC

Xtensa GCC je verzija GNU Compiler Collection-a namijenjena za Xtensa arhitekturu procesora koju ESP32 mikroupravljač koristi. Omogućuje prevođenje C i C++ jezika u strojni kod razumljiv ESP32 kontroleru.

ESP-IDF

ESP-IDF je službeni framework za programiranje svih verzija ESP32 mikroupravljača. Sadrži osnovne biblioteke i upravljače koji se koriste prilikom razvoja softverskih rješenja za ovu arhitekturu, a temelji se na C i C++ programskim jezicima. Projekt je izgrađen na inačici 5.5.5.



Slika 6 - ESP-IDF logo

IMPLEMENTACIJA

Programski dio sustava

Sav programski kod je pisan u VS Code uređivaču koda opremljenom s podrškom za C programski jezik i dodatkom za ESP-IDF, koji uz uređivanje nudi prevođenje, učitavanje na pločicu i praćenje ispisa. Kod je podijeljen u komponente, od kojih svaka ima vlastitu mapu, izvornu datoteku i zaglavlje.

Uporaba koračnih motora

Polukoračna sekvenca kojom se motor okreće i upis jedne faze na ulaze modula:

```
static const uint8_t k_half_step[8] = {0x01, 0x03, 0x02, 0x06,
                                         0x04, 0x0C, 0x08, 0x09};

static void write_phase(stepper_28byj48_handle_t h, uint8_t bits)
{
    for (int i = 0; i < 4; i++) {
        gpio_set_level(h->gpio[i], (bits >> i) & 0x01);
    }
}
```

Zadavanje smjera i brzine u polukoracima po sekundi. Naredba za zaustavljanje gasi sve namote kako se motor ne bi grijao u mjestu:

```
if (direction == STEPPER_28BYJ48_STOP || half_steps_per_second == 0) {
    handle->motion = STEPPER_28BYJ48_STOP;
    write_phase(handle, 0);          // gasi namote
    return;
}
handle->interval_us = 1000000UL / half_steps_per_second;
handle->next_step_us = esp_timer_get_time();
handle->motion = direction;
```

Jedan korak motora, koji se izvodi tek kada prođe zadani razmak između koraka:

```
int64_t now = esp_timer_get_time();
if (now < handle->next_step_us) {
    return false;
}
if (handle->motion == STEPPER_28BYJ48_CW) {
    handle->phase = (uint8_t)((handle->phase + 1) & 0x07);
} else {
    handle->phase = (uint8_t)((handle->phase + 7) & 0x07);
}
write_phase(handle, k_half_step[handle->phase]);
handle->next_step_us += handle->interval_us;
```

Skretanje se dobiva suprotnim smjerovima lijevog i desnog kotača:

```
case CAR_DRIVE_LEFT:
    left  = STEPPER_28BYJ48_CCW;
    right = STEPPER_28BYJ48_CW;
    sps   = s_turn_sps;
    break;
case CAR_DRIVE_RIGHT:
    left  = STEPPER_28BYJ48_CW;
    right = STEPPER_28BYJ48_CCW;
    sps   = s_turn_sps;
    break;
```

Uporaba HC-SR04 senzora

Određivanje priključaka senzora i konstante za pretvorbu trajanja u centimetre:

```
#define HCSR04_TRIG_GPIO 17
#define HCSR04_ECHO_GPIO 16
#define ECHO_TIMEOUT_US  30000
#define US_PER_CM        58.0f
```

Jedno mjerenje: impuls na TRIG pinu, zatim mjerenje trajanja visokog stanja na ECHO pinu:

```
gpio_set_level(s_pins.trig_gpio, 1);
esp_rom_delay_us(10);
gpio_set_level(s_pins.trig_gpio, 0);

int64_t rise = wait_for_level(1, ECHO_TIMEOUT_US);
int64_t fall = wait_for_level(0, ECHO_TIMEOUT_US);
if (rise < 0 || fall < 0) {
    return ESP_ERR_TIMEOUT;
}
*distance_cm = (float)(fall - rise) / US_PER_CM;
```

Kako se zvuk zna odbiti od bočnog zida, uzimaju se tri uzastopna mjerenja, a kao rezultat se uzima srednji po veličini:

```
for (int i = 0; i < 3; i++) {
    if (hcsr04_measure_cm(&samples[taken]) == ESP_OK) {
        taken++;
    }
    vTaskDelay(pdMS_TO_TICKS(15));
}
sort3(&samples[0], &samples[1], &samples[2]);
*distance_cm = samples[1];           // medijan
```

Uporaba web sučelja

Podizanje vlastite Wi-Fi pristupne točke, zaštićene WPA2 lozinkom:

```
wifi_config_t ap = {
    .ap = {
        .channel = 1,
        .max_connection = 4,
        .authmode = WIFI_AUTH_WPA2_PSK,
    },
};
ESP_ERROR_CHECK(esp_wifi_set_mode(WIFI_MODE_AP));
ESP_ERROR_CHECK(esp_wifi_set_config(WIFI_IF_AP, &ap));
ESP_ERROR_CHECK(esp_wifi_start());
```

Putanje poslužitelja: jedna vraća stranicu, druga prima naredbu vožnje, a treća vraća trenutno stanje sustava:

```
const httpd_uri_t routes[] = {
    {.uri = "/", .method = HTTP_GET, .handler = index_get},
    {.uri = "/api/drive", .method = HTTP_GET, .handler = drive_get},
    {.uri = "/api/status", .method = HTTP_GET, .handler = status_get},
};
```

Naredba se šalje kao jedno slovo, a poslužitelj prihvaća samo slova koja stvarno postoje:

```
httpd_query_key_value(query, "cmd", value, sizeof(value));
command = value[0];

if (command && strchr("FBLRSZ", command)) {
    s_command_cb(command);
    return httpd_resp_sendstr(req, "OK");
}
```

Implementacija logike sustava

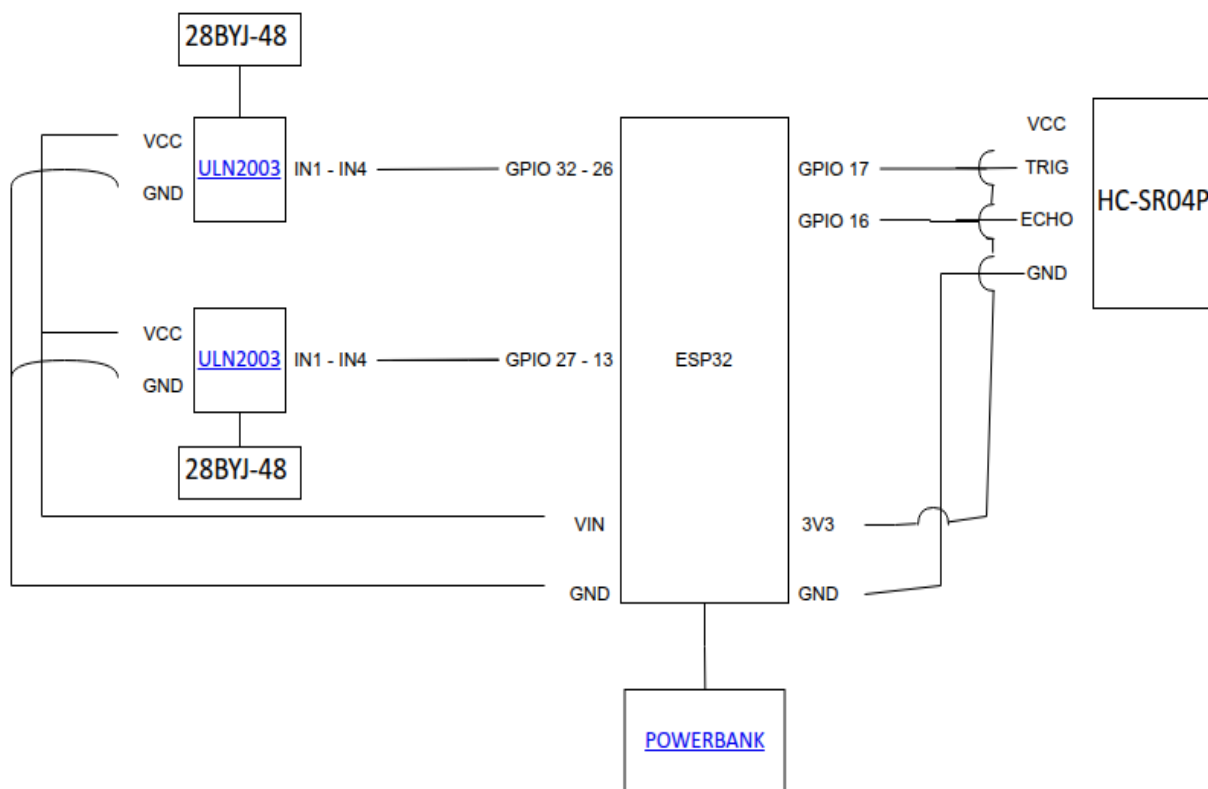
Naredba se preuzima svakih dvadeset milisekundi i, ako je iza autića prepreka bliža od dvadeset centimetara, a zatražena je vožnja natrag, zamjenjuje se zaustavljanjem:

```
while (true) {
    car_drive_motion_t wanted = s_user_request;

    if (s_obstacle && wanted == CAR_DRIVE_BACKWARD) {
        wanted = CAR_DRIVE_STOP;    // vozi natrag ne, naprijed i okret da
    }

    car_drive_set_motion(wanted);
    vTaskDelay(pdMS_TO_TICKS(CONTROL_PERIOD_MS));
}
```

Elektronička shema sustava



Slika 7 - Elektronička shema sustava

LITERATURA

<https://docs.espressif.com/projects/esp-idf/en/stable/esp32/get-started/>

<https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/>

<https://www.freertos.org/Documentation/>

<https://www.upesy.com/blogs/tutorials/esp32-pinout-reference-gpio-pins-ultimate-guide>

<https://www.alldatasheet.com/>